

Mini-Projet — Vision par Ordinateur Moderne

Classification d'images, transfer learning et interprétabilité

Vision par Ordinateur — ESIEE-IT Pontoise

Master 2 E3IN — IA & Big Data

Livrables

- **Notebook Jupyter reproductible** exécuté en local ou sur Google Colab.
- **Rapport technique rédigé en $\text{\LaTeX}$** de 4 à 8 pages présentant la méthodologie, les expérimentations, les résultats obtenus et l'analyse critique.
- **Rapport individuel rédigé en $\text{\LaTeX}$** de 2 à 4 pages expliquant la contribution personnelle, les tâches réalisées et les choix techniques effectués.

Contexte et objectifs

La vision par ordinateur moderne ne consiste plus uniquement à classer une image. Un système réel doit être capable d'extraire des représentations visuelles robustes, de généraliser à de nouvelles situations, d'expliquer ses décisions et de fonctionner sous différentes contraintes de données, de calcul et de fiabilité.

Dans ce module, plusieurs notions ont été étudiées : représentation numérique des images, convolution, CNN, pooling, dropout, normalisation, transfer learning, architectures pré-entraînées, interprétabilité, biais et corrélations fallacieuses.

L'objectif de ce mini-projet est de construire une étude complète de classification d'images sur un dataset réel à l'aide d'un modèle pré-entraîné et d'outils d'interprétabilité.

Les expérimentations permettront d'analyser :

- L'intérêt du transfer learning pour un problème de classification d'images.
- La différence entre feature extraction et fine-tuning léger.
- L'impact du prétraitement et de la data augmentation.
- Les erreurs de classification et les confusions entre classes.
- Les zones de l'image utilisées par le modèle pour prendre une décision.
- Les limites des modèles de vision face aux biais et aux corrélations fallacieuses.

Documentation utile

- PyTorch : <https://pytorch.org/>.
- Torchvision : <https://pytorch.org/vision/stable/index.html>.
- Torchinfo : <https://github.com/TylerYep/torchinfo>.

- scikit-learn : <https://scikit-learn.org/stable/>.
- Matplotlib : <https://matplotlib.org/>.
- Grad-CAM PyTorch : <https://github.com/jacobgil/pytorch-grad-cam>.
- Intel Image Classification Dataset : <https://www.kaggle.com/datasets/puneet6060/intel-image-classification>.

1 Dataset

Le dataset utilisé est **Intel Image Classification**. Il contient plusieurs catégories de scènes naturelles :

- Buildings.
- Forest.
- Glacier.
- Mountain.
- Sea.
- Street.

Ce dataset est intéressant car certaines classes présentent des similarités visuelles importantes et des arrière-plans complexes.

Travail demandé :

1. Charger le dataset.
2. Afficher plusieurs exemples d'images par classe.
3. Vérifier la taille des images.
4. Vérifier le nombre d'images par classe.
5. Afficher la distribution des classes.
6. Vérifier si le dataset est équilibré.
7. Afficher la forme des tenseurs manipulés.
8. Expliquer le rôle des dimensions C , H et W .

Questions d'analyse :

- Certaines classes semblent-elles visuellement proches ?
- Quels types de confusions peut-on anticiper avant l'entraînement ?
- Les arrière-plans risquent-ils d'influencer les prédictions ?
- Certaines classes semblent-elles plus difficiles à reconnaître ?
- Le dataset semble-t-il représentatif d'un contexte réel ?

2 Partie 1 — Pipeline de prétraitement

Dans cette partie, l'objectif est de construire un pipeline complet de préparation des données.

Travail demandé :

1. Convertir les images en tenseurs PyTorch.
2. Redimensionner les images en fonction du modèle utilisé.
3. Normaliser les images.

4. Créer les ensembles train, validation et test.
5. Créer les DataLoaders.
6. Mettre en place une stratégie simple de data augmentation.
7. Afficher plusieurs images avant et après data augmentation.

Transformations imposées :

- RandomHorizontalFlip.
- RandomRotation.
- RandomCrop.

Questions d'analyse :

- Pourquoi normaliser les images avant l'entraînement ?
- Pourquoi redimensionner les images ?
- Pourquoi la data augmentation est-elle utile ?
- Certaines augmentations peuvent-elles modifier le sens de l'image ?
- Pourquoi faut-il éviter d'appliquer ces transformations au test set ?

3 Partie 2 — Feature extraction avec ResNet18

Le backbone utilisé est **ResNet18 pré-entraîné sur ImageNet**. Il est utilisé comme extracteur de caractéristiques.

Travail demandé :

1. Charger ResNet18 pré-entraîné.
2. Adapter la couche finale au nombre de classes du dataset.
3. Geler le backbone.
4. Entraîner uniquement la couche finale.
5. Afficher l'architecture du modèle.
6. Afficher le nombre total de paramètres.
7. Afficher le nombre de paramètres entraînaibles.
8. Tracer les courbes de loss et d'accuracy.

Questions d'analyse :

- Pourquoi un modèle pré-entraîné peut-il être utile ?
- Pourquoi les premières couches peuvent-elles être réutilisées ?
- Quel est l'intérêt de geler le backbone ?
- La feature extraction est-elle suffisante pour ce dataset ?
- Le modèle semble-t-il généraliser correctement ?

4 Partie 3 — Fine-tuning léger

Dans cette partie, l'objectif est d'adapter plus finement ResNet18 au dataset choisi.

Le fine-tuning reste léger afin de limiter le coût de calcul et le risque d'overfitting.

Travail demandé :

1. Dégeler uniquement les dernières couches du modèle.
2. Utiliser un learning rate faible.
3. Entraîner le modèle sur quelques époques.
4. Comparer les résultats avec la feature extraction.
5. Comparer le temps d'entraînement.
6. Comparer les courbes de loss et d'accuracy.

Questions d'analyse :

- Pourquoi ne faut-il pas forcément dégeler tout le modèle ?
- Pourquoi utiliser un faible learning rate ?
- Le fine-tuning améliore-t-il les performances ?
- Le fine-tuning augmente-t-il le risque d'overfitting ?
- Dans quel cas la feature extraction suffit-elle ?

5 Partie 4 — Évaluation des modèles

1. Évaluer le modèle en feature extraction sur le test set.
2. Évaluer le modèle fine-tuné sur le test set.
3. Calculer l'accuracy globale.
4. Calculer la precision, le recall et le F1-score par classe.
5. Afficher la matrice de confusion.
6. Identifier les classes les mieux reconnues.
7. Identifier les classes les plus difficiles.
8. Comparer les deux approches.

Questions d'analyse :

- L'accuracy suffit-elle pour juger la qualité du modèle ?
- Quelles classes sont les plus confondues ?
- Les confusions observées étaient-elles prévisibles ?
- Le fine-tuning améliore-t-il toutes les classes ou seulement certaines ?
- Quel modèle choisiriez-vous dans un contexte réel ?

6 Partie 5 — Étude des erreurs : fil rouge du mini-projet

Cette partie est centrale dans le projet.

Sélectionner au minimum 8 erreurs de classification obtenues avec le modèle en feature extraction. Ces mêmes exemples devront être réanalysés avec le modèle fine-tuné.

Travail demandé :

Pour chacune des 8 erreurs sélectionnées :

1. Afficher l'image.
2. Indiquer la vérité terrain.
3. Indiquer la prédiction du modèle en feature extraction.
4. Indiquer la prédiction du modèle fine-tuné.
5. Indiquer si l'erreur est corrigée par le fine-tuning.
6. Proposer une hypothèse expliquant l'erreur.

Types d'erreurs possibles :

- Similarité visuelle entre deux classes.
- Objet partiellement visible.
- Arrière-plan trompeur.
- Mauvais contraste.
- Corrélation fallacieuse.
- Ambiguïté même pour un humain.

7 Partie 6 — Interprétabilité avec Grad-CAM

Grad-CAM permet d'analyser les zones importantes de l'image utilisées par le modèle pour produire ses prédictions.

Travail demandé :

1. Appliquer Grad-CAM sur plusieurs bonnes prédictions.
2. Appliquer Grad-CAM sur plusieurs erreurs.
3. Visualiser les zones importantes de l'image.
4. Réanalyser au moins 4 erreurs sélectionnées dans la partie précédente.
5. Expliquer si les zones activées semblent pertinentes.

Questions d'analyse :

- Le modèle regarde-t-il réellement l'objet principal ?
- Les zones importantes sont-elles cohérentes avec la classe prédite ?
- Certaines prédictions correctes reposent-elles sur de mauvais indices ?
- Certaines erreurs sont-elles explicables par l'attention portée à l'arrière-plan ?
- Grad-CAM permet-il de prouver que le modèle a raison ?

8 Partie 7 — Comparaison finale

Construire un tableau de synthèse des approches testées.

Approche	Métriques	Forces	Limites
Feature extraction	Accuracy, F1-score	Rapide, peu coûteux	Adaptation limitée
Fine-tuning léger	Accuracy, F1-score	Meilleure adaptation au dataset	Plus coûteux, risque d'overfitting

Compléter ce tableau avec :

- Le temps d'entraînement estimé.
- Le nombre de paramètres entraînables.
- Le risque d'overfitting.
- Le modèle recommandé en pratique.

9 Analyse critique

- Un modèle de vision comprend-il réellement une image ? Justifier.
- Pourquoi un modèle peut-il obtenir une bonne accuracy sans apprendre les bons indices visuels ?
- Quelle différence existe-t-il entre corrélation statistique et compréhension visuelle ?
- Le transfer learning résout-il toutes les limites des petits datasets ?
- Pourquoi certaines erreurs persistent-elles malgré le fine-tuning ?
- Quels biais ou corrélations fallacieuses avez-vous observés ?
- Pourquoi l'interprétabilité est-elle importante dans les domaines sensibles ?
- Quel compromis choisir entre précision, rapidité, coût de calcul et explicabilité ?